

KI Service
Zentrum
by Hasso-Plattner-Institut

HPI

WORK SHOP

REINFORCEMENT
LEARNING II

DESIGNING
AND BUILDING
RL ENVIRONMENTS

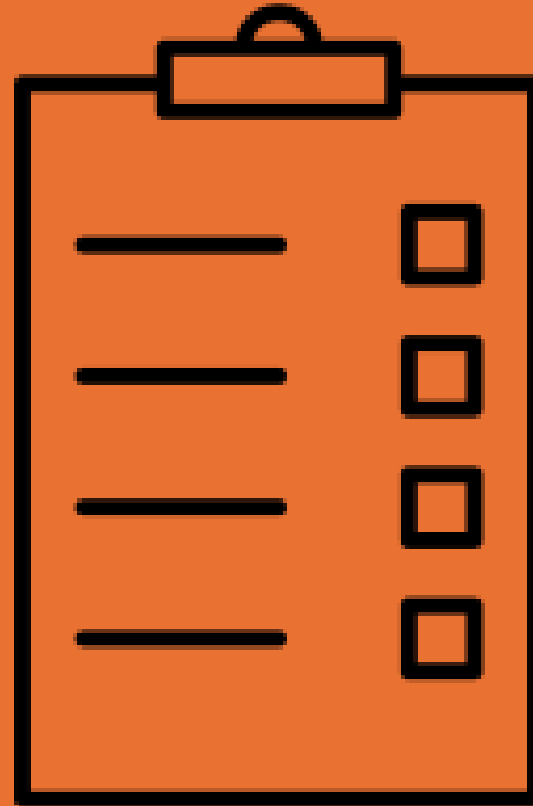
AI

FUNDED BY



Federal Ministry
of Research, Technology
and Space

Attendance List



With funding from the:



Federal Ministry
of Research, Technology
and Space

Overview of the AI Service Center

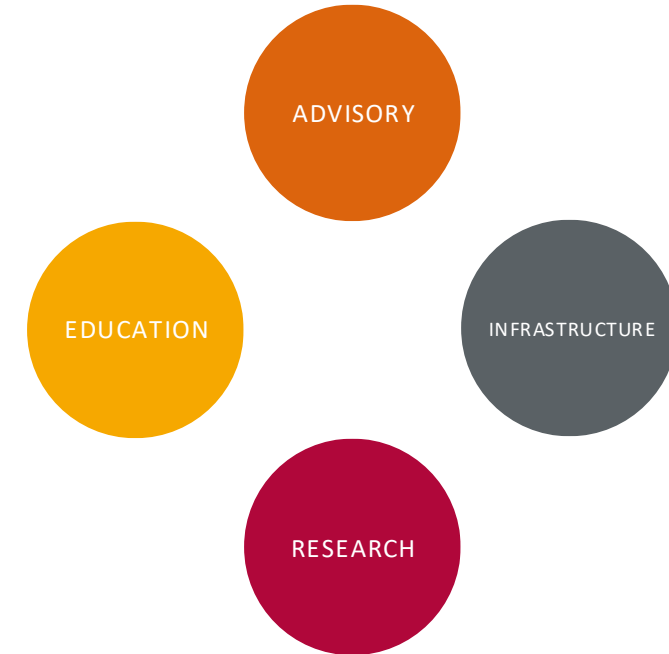
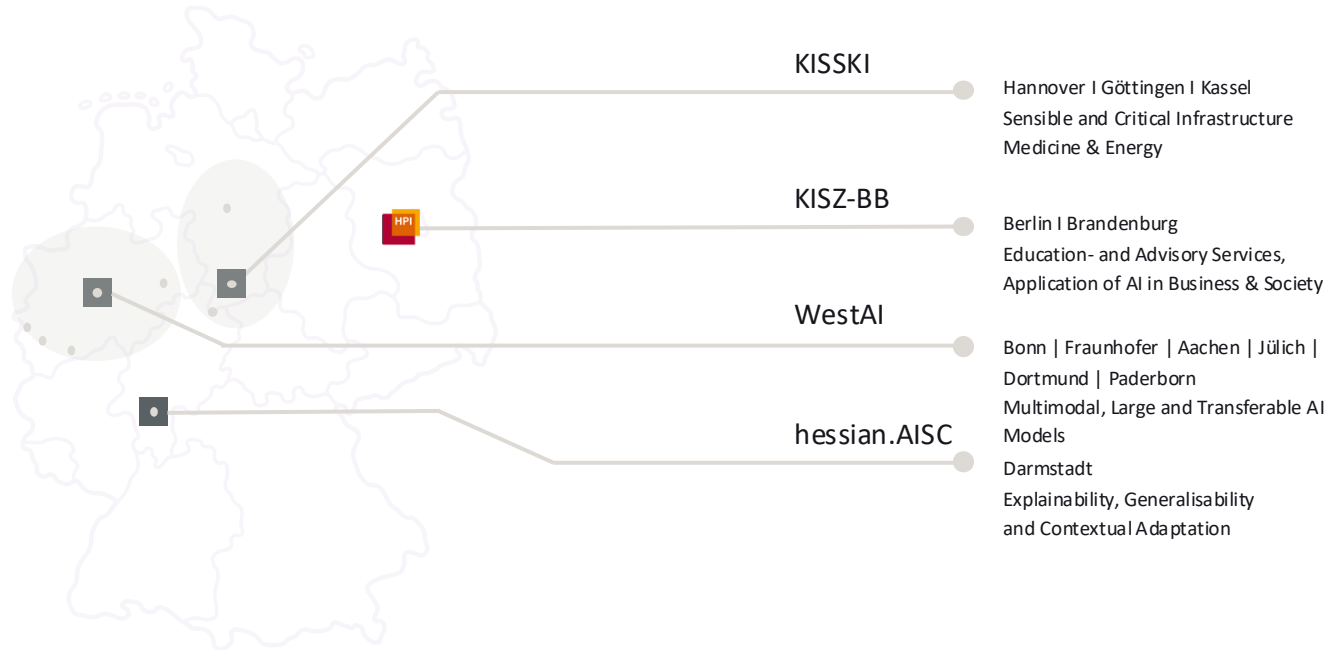


With funding from the:



Federal Ministry
of Research, Technology
and Space

AI-Services @ HPI



SPONSORED BY THE

EDUCATION & TRAINING

Talks

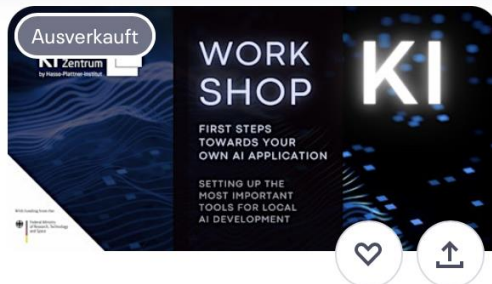
Research, innovation-oriented guest talks

Work shops

- On various, hands-on topics: People in front of a laptop
- Accompanied by introductory videos
- Example topics: speech2summary, Docker, semantic search

MOOCs

- Down the road: 3-10 MOOCs
- Example topic: “ChatGPT: What does generative AI mean for our society?”



First Steps Towards Your Own AI Application

Donnerstag um 15:00

Hasso-Plattner-Institut

Kostenlos

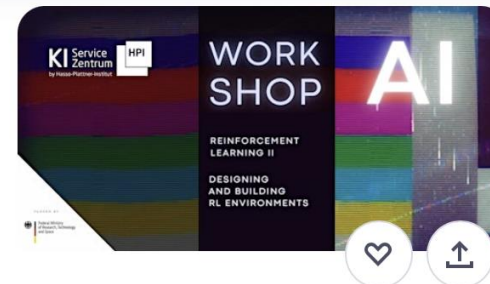


Effizientes Abfragen von Informationen – Teil II

Mi. 25. Feb., 13:00

Hasso -Plattner-Institut

Kostenlos



Reinforcement Learning – Part II

Do. 26. Feb., 10:00

Hasso-Plattner-Institut

Kostenlos



Erste Schritte zu Ihrer eigenen KI-Anwendung

Fr. 13. März, 13:00

Hasso -Plattner-Institut

Kostenlos



Guest Talk: Automatische Deepfake-Erkennung mit KI

Di. 17. März, 17:00

Hasso -Plattner-Institut

Kostenlos

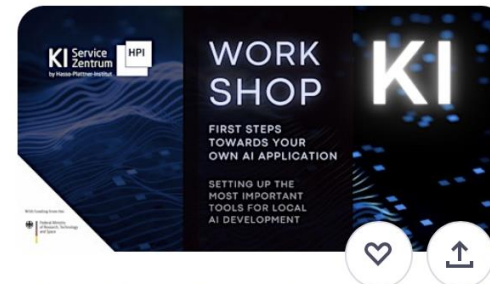


Effizientes Abfragen von Informationen – Teil III

Do. 26. März, 14:00

Hasso -Plattner-Institut

Kostenlos

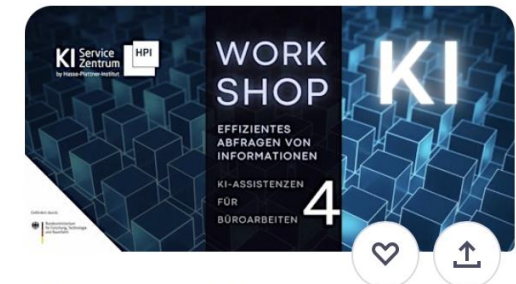


First Steps Towards Your Own AI Application

Mo. 13. Apr., 10:00

Hasso-Plattner-Institut

Kostenlos



Effizientes Abfragen von Informationen – Teil IV

Di. 28. Apr., 15:00

Hasso -Plattner-Institut

Kostenlos

ADVISORY

AI Advisory

- General Questions regarding AI,
- Computing Infrastructure,
- Model, Architecture, Framework Selection

AI Pilot Projects

1. Understanding and Identifying the Use Case
2. Specifying the Use Case
3. Developing a Solution
4. Finalizing and Documenting

Cooperations

- Co-organized Network Meetings or Workshops

Start-up Support

- All of the above
- Support with Networking

HPI ECOSYSTEM





Training

- 8 x 8 NVIDIA H100 GPU

Inference

- 5 x 8 NVIDIA A30 GPU

ARM Server

- Ampere Altra Max M128-30 CPU
- 2 x NVIDIA L40 GPUs

GPU Server

- AMD Epyc CPU
- 8 x NVIDIA L40S GPU

Edge

- ARMv8 CPU
- NVIDIA Jetson AGX Module

Storage

- 1.5 PB NVRAM

Network

- 400 Gb/s Infiniband
- 200 Gb/s Ethernet

Personal Introduction



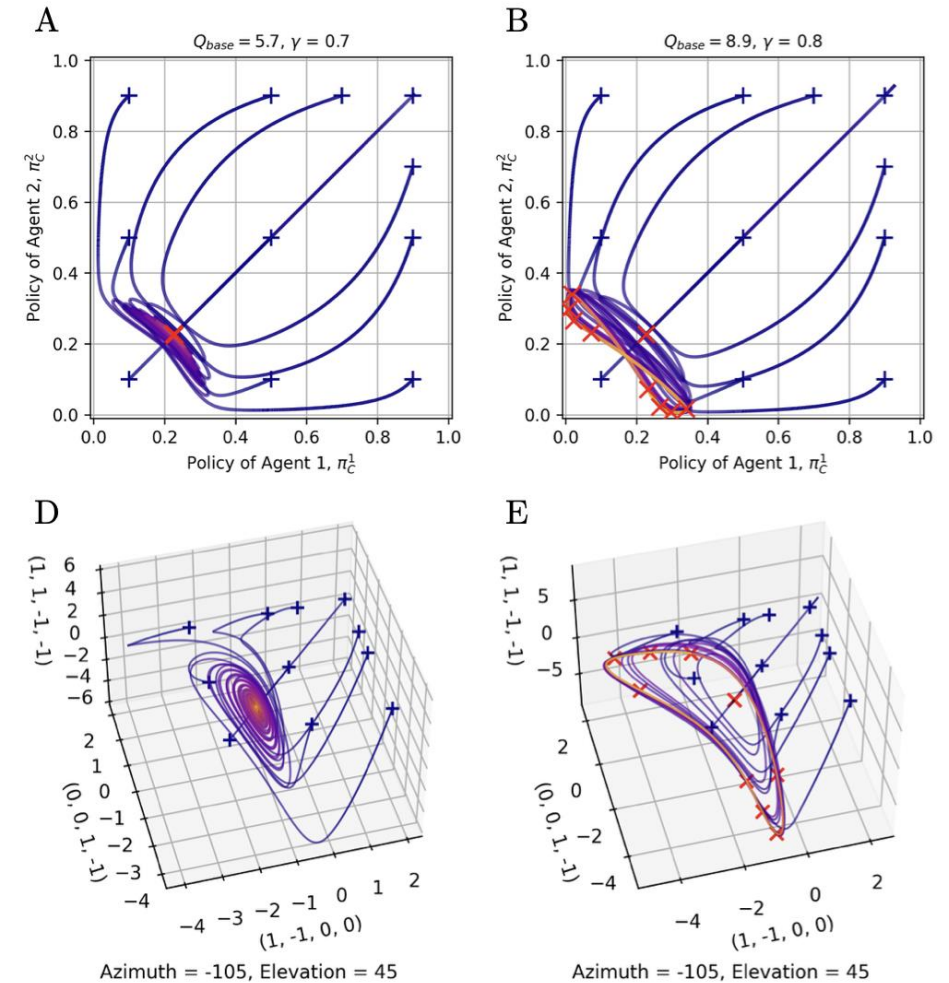
With funding from the:



Federal Ministry
of Research, Technology
and Space

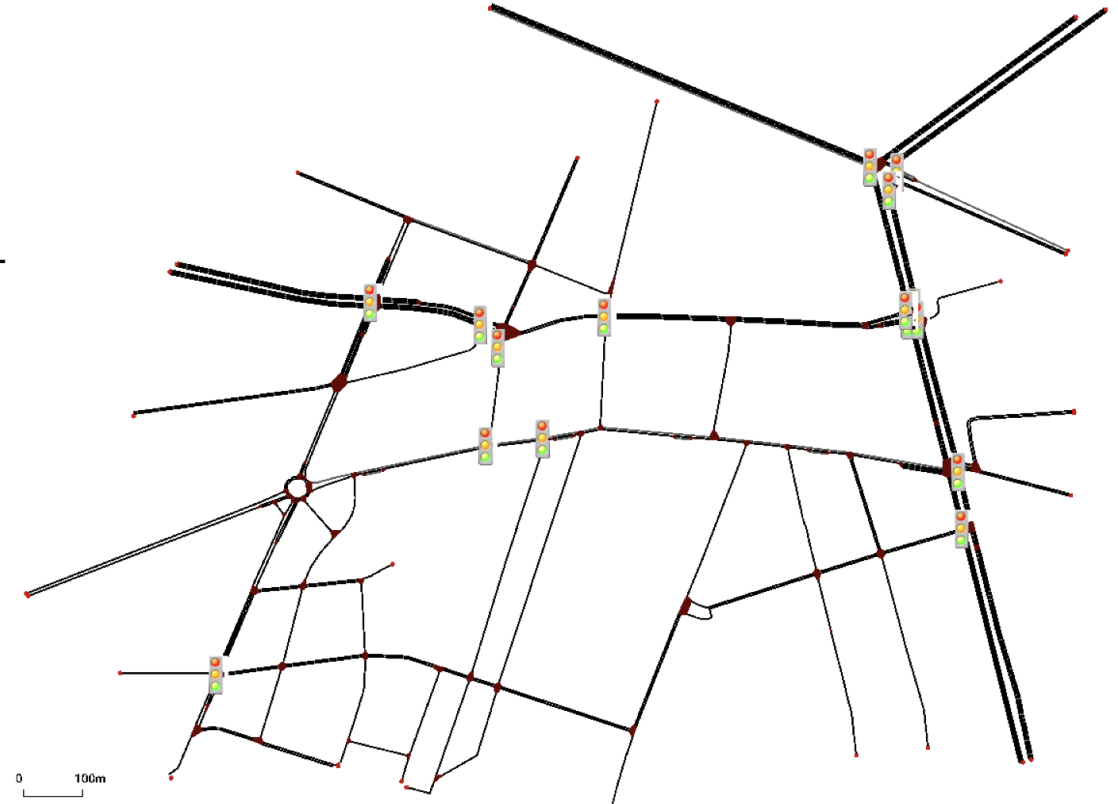
Personal Introduction – David Goll

- Background in Physics
- **Master's thesis:**
"Analysis of Multi-Agent Reinforcement Learning from a Statistical Physics Perspective"
<https://arxiv.org/abs/2501.00160>
- **Main message:**
 Complex Dynamics can arise even in simple MARL systems



Personal Introduction – Jill Barvencik

- Background in Mechanical Engineering and Computational Engineering Science
- **Master's thesis:** "Deep Multi-Agent Reinforcement Learning for Real-World Large-Scale Traffic"
- **Used RL to stabilize seq2seq training and improve robustness** beyond sparse reward signals



Personal Introduction – Your turn

If you like to, please introduce yourself:

- What is your background?
- Why are you interested in RL?
- Are there any usecases where you would like to use RL?

Workshop Intro



With funding from the:



Federal Ministry
of Research, Technology
and Space

Content Overview

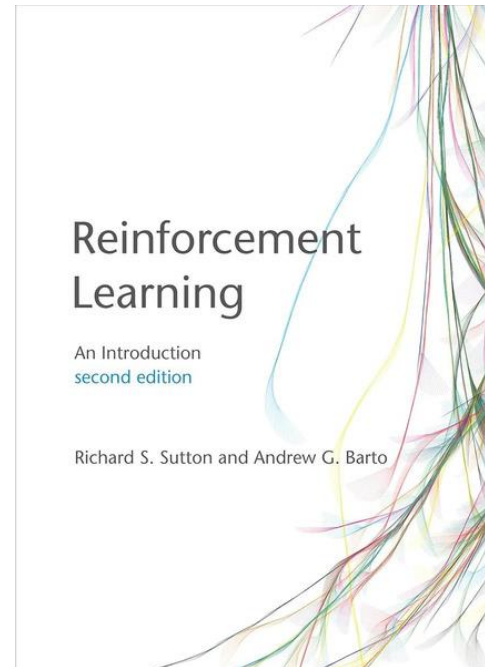
- Recap RL basics
- Problem statement
- Install repository
- Design your own policy
- Is this a RL problem?
- How to design it as a RL problem
 - Gymnasium API, Live coding, design choices, choice of algorithm
- Is this a RL problem? (Part II)
 - Recap
 - Examples of real life RL usage
- Conclusion

With funding from the:

Motivation and context |

“Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do.” - Richard Sutton, Andrew Barto

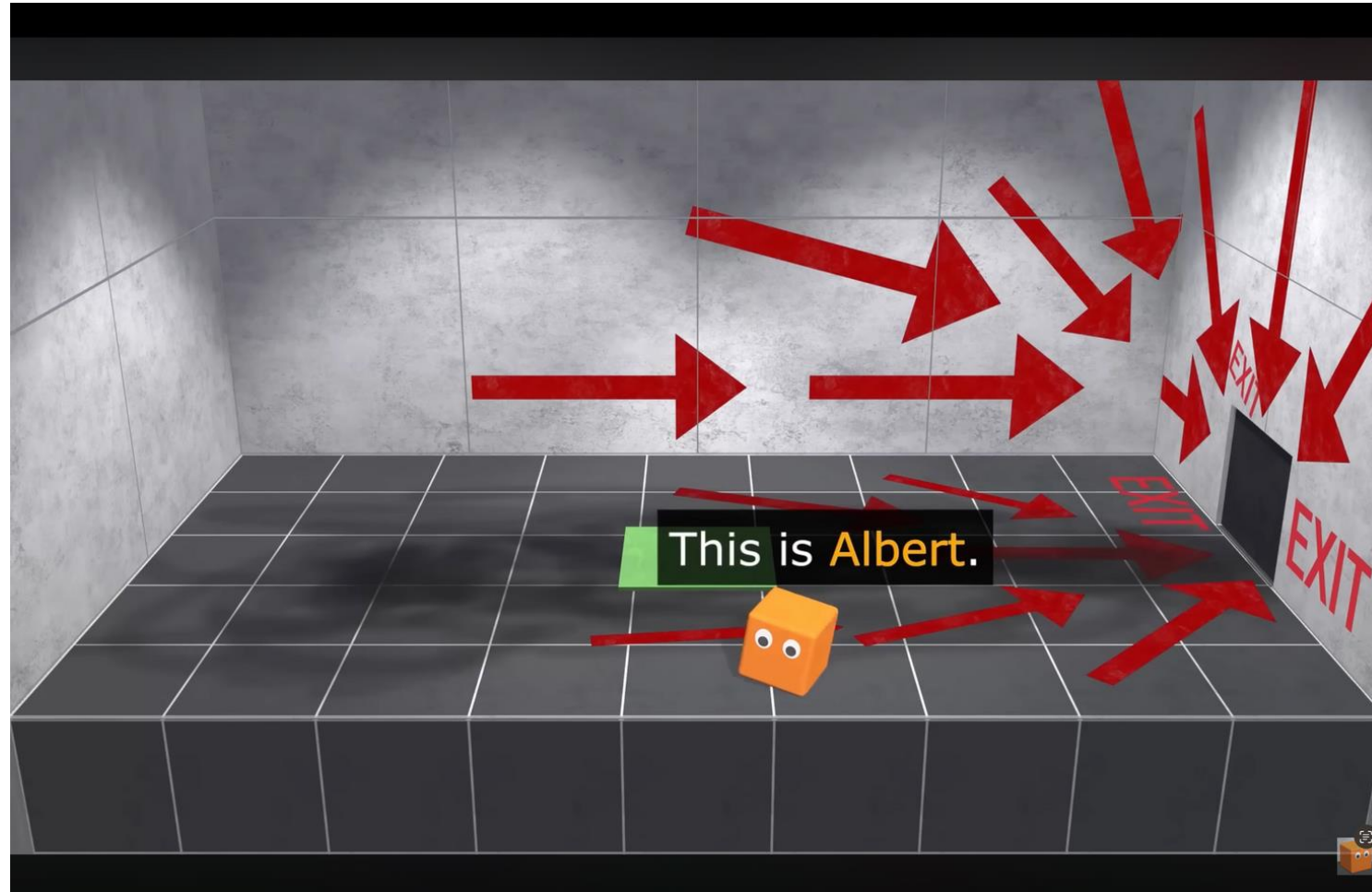
kisz@hpi.de
hpi.de/kisz



Motivation and context |

“Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do.” -Richard S. Sutton

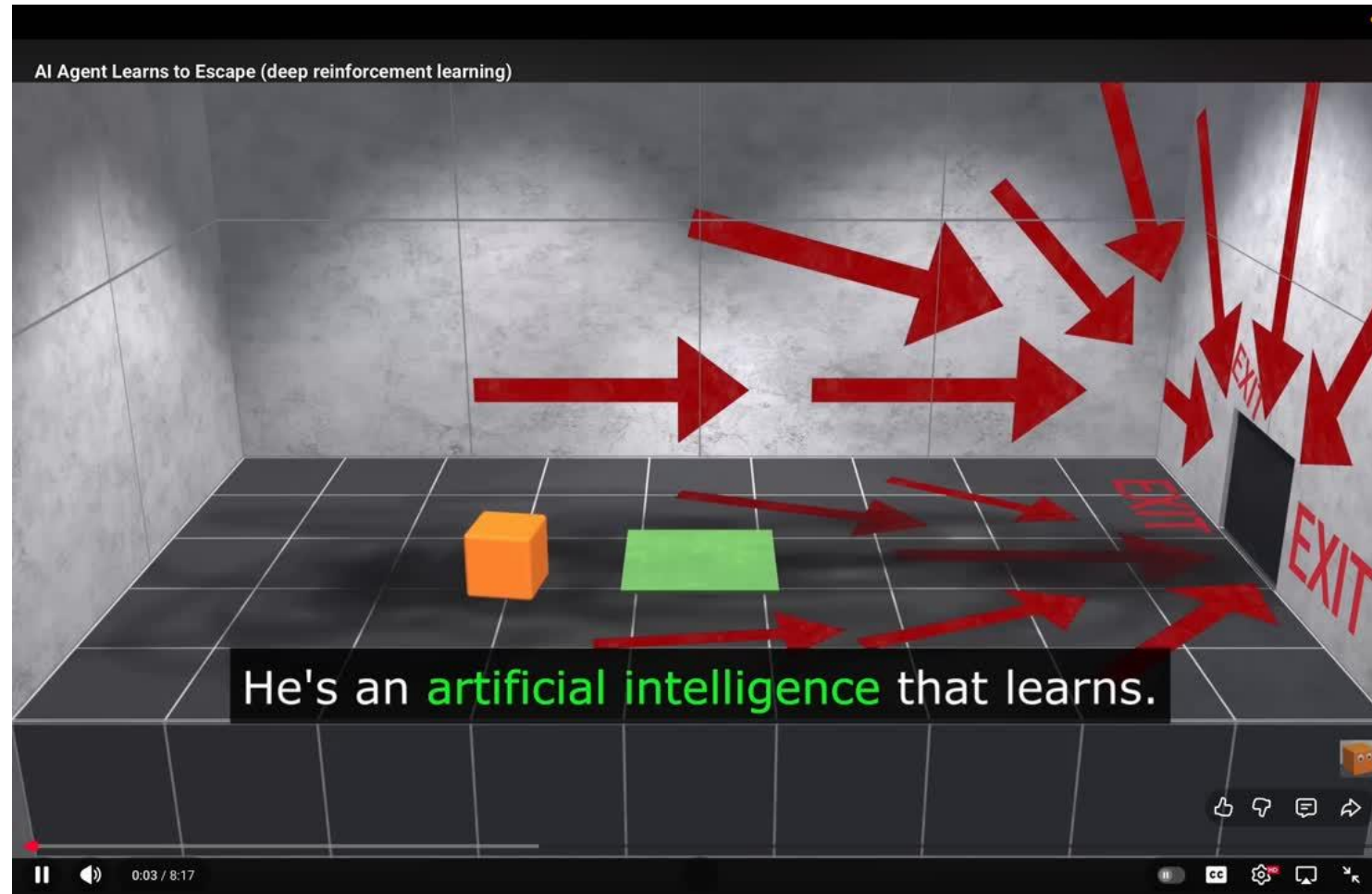
kisz@hpi.de
hpi.de/kisz



Motivation and context |

“Of all the forms of machine learning, reinforcement learning is the closest to the kind of learning that humans and other animals do.” -Richard S. Sutton

kisz@hpi.de
hpi.de/kisz

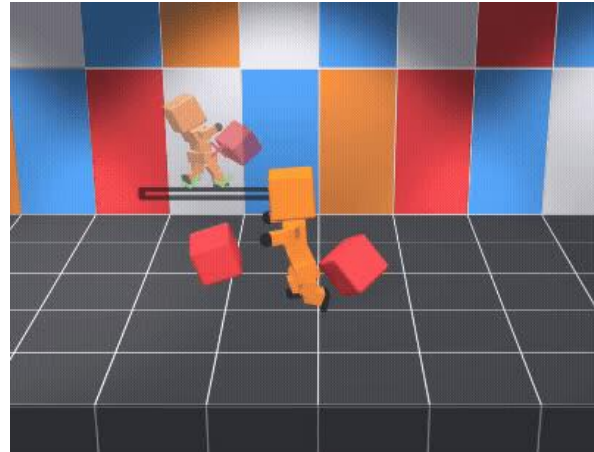


Motivation and context | | : Applications

kisz@hpi.de

hpi.de/kisz

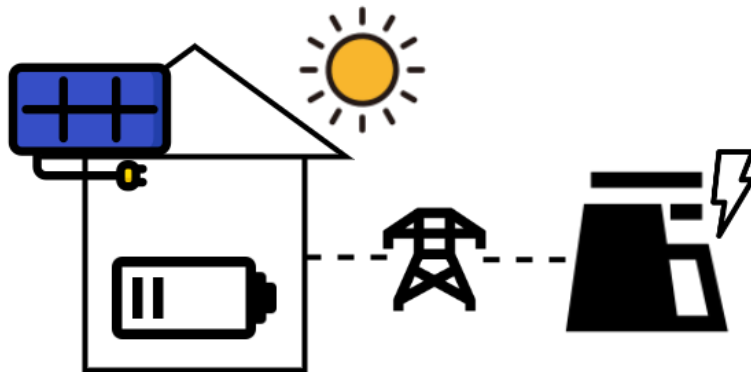
Robotics



AI Warehouse

https://www.youtube.com/watch?v=L_4BPjLBF4E&t=60s

Control Energy Systems



<https://github.com/tobirohrer/building-energy-storage-simulation>

RLHF for Chatbots

What is the biggest animal in the world?



Wähle die beste Antwort

User-Auswahl

Modell-Präferenz

Beide

Score: 0.007

Tiger.

seed: 11

Score: 0.016

The biggest animal in the world is a blue whale weighing up to 16,000 pounds (7,300 kg).

seed: 22

Score: 0.007

Tiger.

seed: 33

Keine Antwort ist gut



Danke für dein Feedback!

Motivation and context |||

Supervised Learning

- Models are **trained on labeled data** to predict outputs

Reinforcement Learning

- Learning optimal behavior by **interacting with an environment**

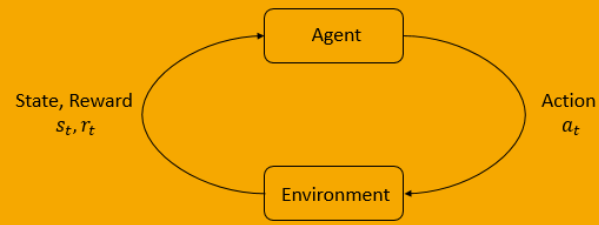
Unsupervised Learning

- Learn to **identify previously unknown patterns** and relationships in unlabeled data

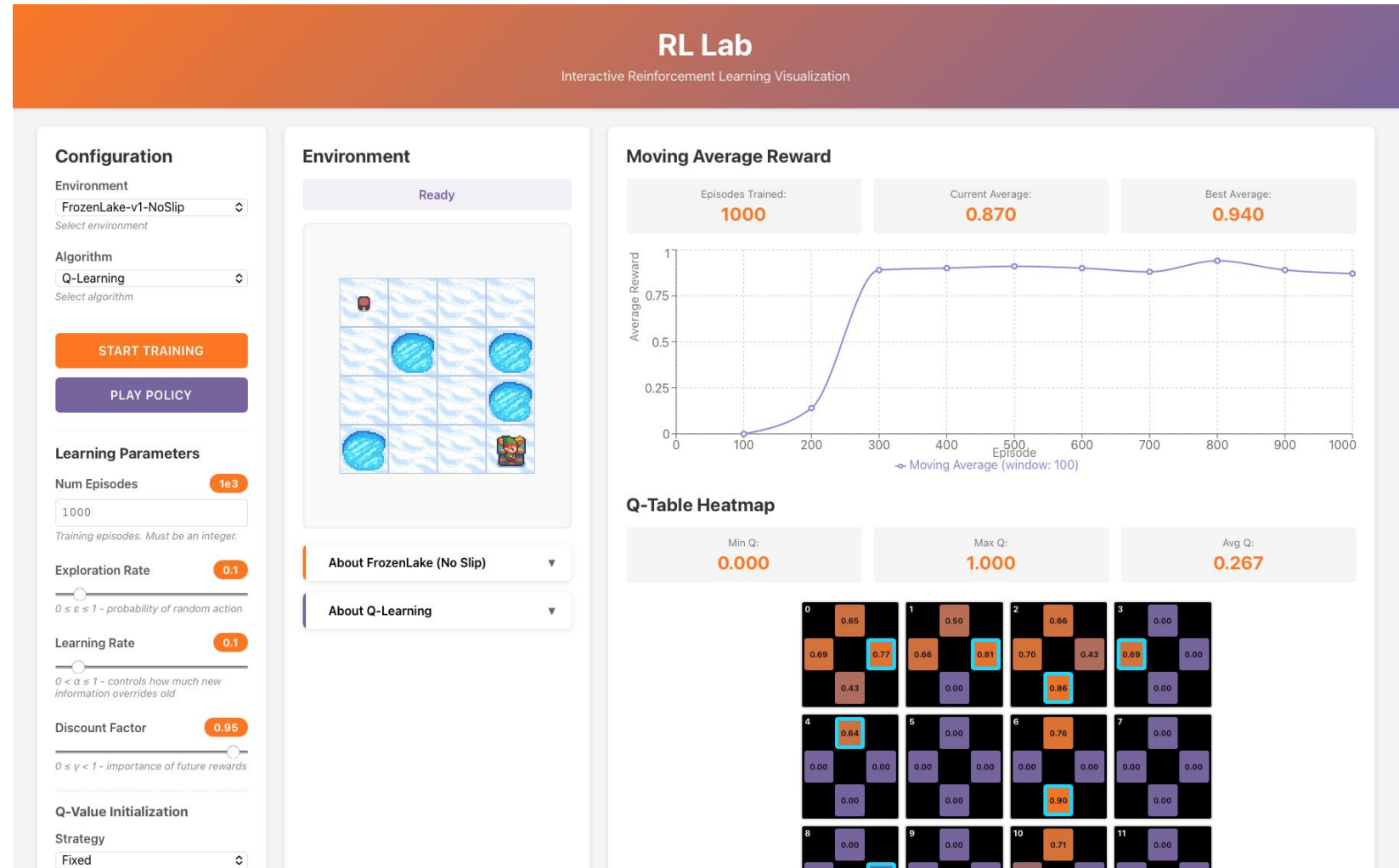


used when **optimal actions are too complex or unknown to define** but can be learned through experience

Recap RL basics

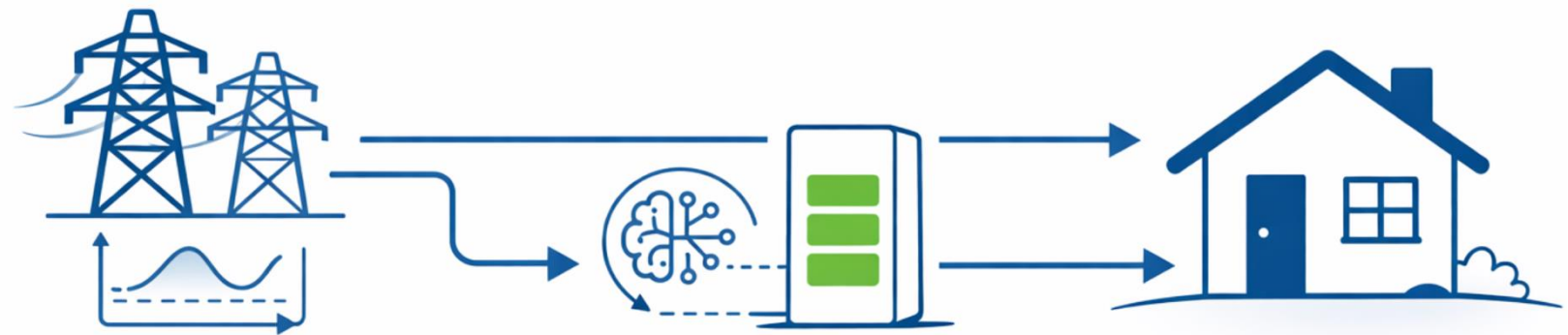


With funding from the:



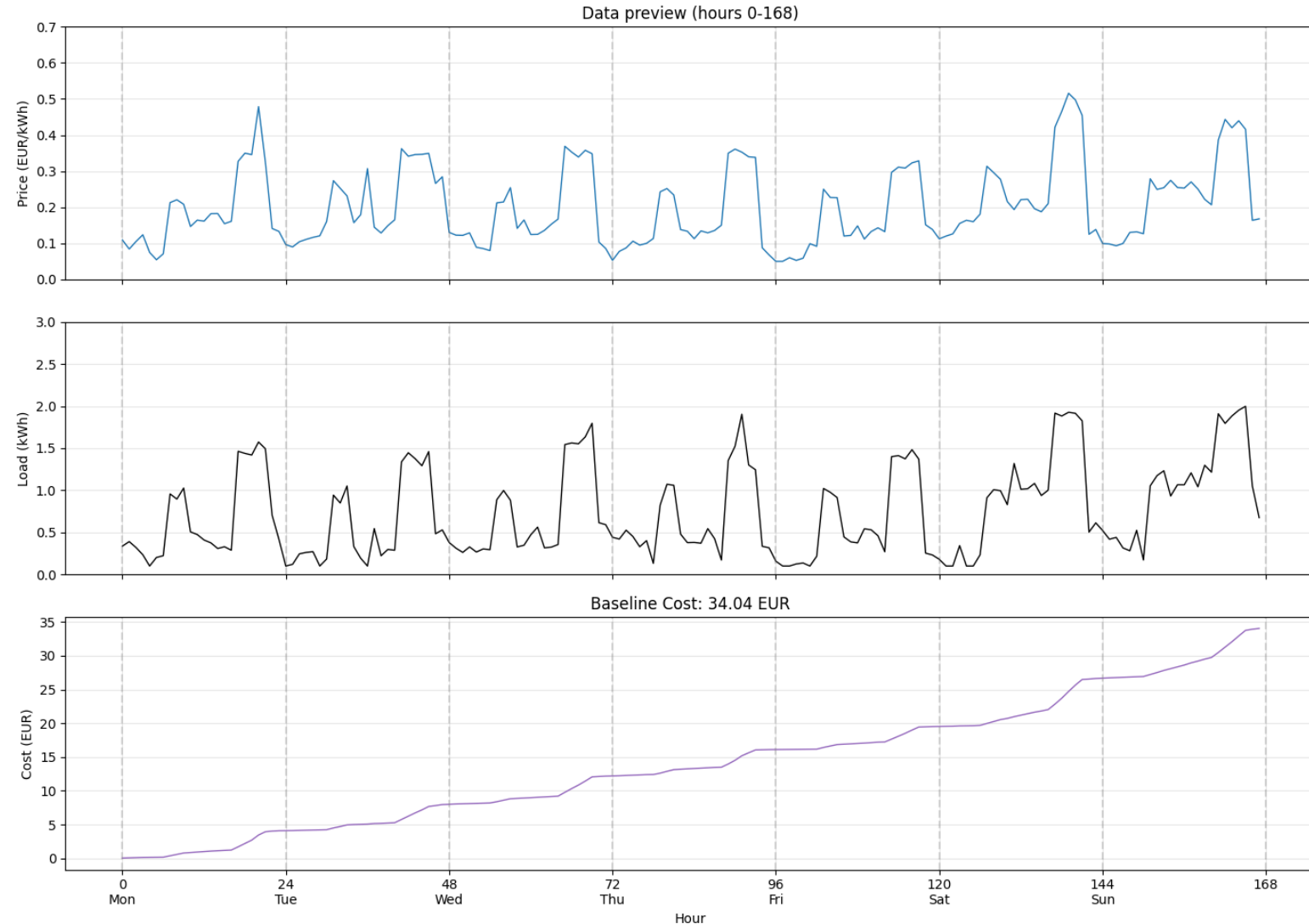
Problem statement

Imagine your home has a battery that can store electricity for later use. Since electricity prices and demand change throughout the day, you must decide when to charge and when to discharge the battery. The goal is to find a strategy that minimizes monthly energy costs.



With funding from the:

Data preview



With funding from the:



What we will work with today

workshop-rl2-implementation

00_explore_data.ipynb

01_workshop > notebooks > 00_explore_data.ipynb > M Exploring the Dataset

Generate + Code + Markdown | Run All Restart Clear All Outputs ... workshop-rl2-implementation (Python 3.13.7)

Exploring the Dataset

This notebook introduces the electricity price and household load data used throughout the workshop.

The data is synthetic (see `scripts/generate_data.py`) with daily and weekly patterns:

- **87,600 hours** of data (10 years, hourly resolution)
- **Prices** in €/kWh — vary by time of day and weekday/weekend
- **Loads** in kWh/h — household consumption patterns

Please note that there are no periodic patterns in the data besides the daily and weekly cycles.

We have kept the data simple to focus on core concepts.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # The dataset contains primarily two timeseries: electricity prices and loads of the household:
5 prices = np.load('../03_data/prices.npy')
6 loads = np.load('../03_data/loads.npy')

```

With funding from the:



Federal Ministry
of Research, Technology
and Space

Install Repository

<https://github.com/aihpi/workshop-rl2-implementation>



guest wifi

With funding from the:



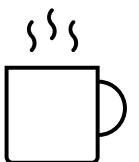
Gefördert durch:



Workshop: Reinforcement Learning II - Implementation

This repository hosts the materials of the Reinforcement Learning II workshop of the AI Service Center Berlin Brandenburg. In part two of this workshop series, we use a simple illustrative example, such as managing an energy storage system, to demonstrate the step-by-step process of translating a given task into a RL setup.

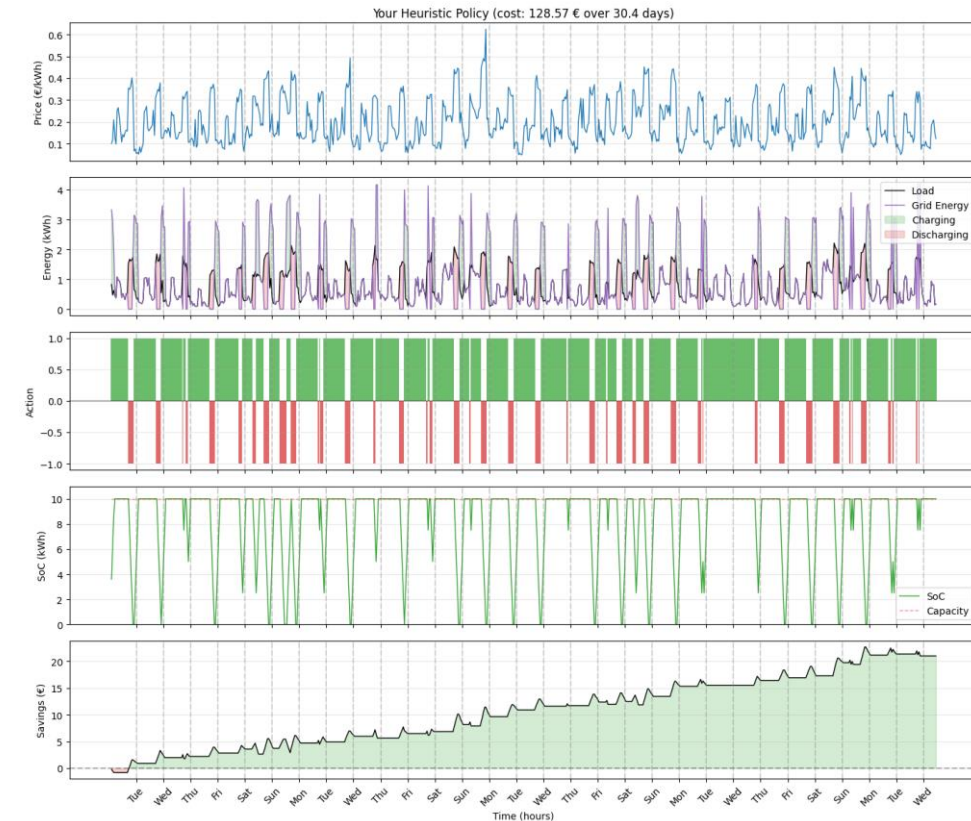
The Problem



If you are done, take a look at the 00_explore_data.ipynb notebook or take a break

Design your own policy!

- What would be the policy you would choose, looking at the data?
- Work through the 01_design_heuristic_policy.ipynb notebook
- What is the best you can achieve?
- 5 – 10 min



No battery: 149.60 €
 Your heuristic: 128.57 €
 Savings: +21.03 € (+14.1%)

With funding from the:

Is this a RL problem?

When RL could be useful:

- **Simulation / safe exploration is feasible**

RL is data hungry and learns by trial-and-error!

- **Sequential decision-making**

Decisions affect the environment, requiring long-term planning

- **Complex or unknown dynamics**

System is too difficult to optimize analytically

→ Linear Programming / Model Predictive Control can actually solve this!

With funding from the:

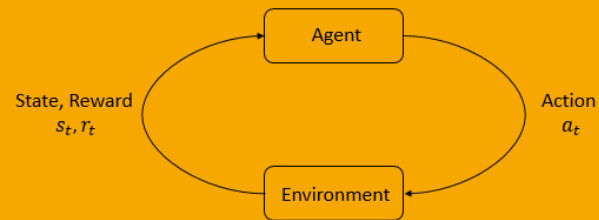
Baselines to test against

- **Linear Programming (LP)**
 - Optimizes a linear objective function subject to linear constraints
 - Finds optimal solution if given perfect knowledge (complete data of episode)
- **Model Predictive Control (MPC)**
 - Repeatedly optimizes over a rolling time horizon (e.g. with LP)
 - At each step: solve optimization, execute first action, re-plan with updated info

→ LP and MPC will serve us as baselines to compare our RL agent against

How to design it as a RL problem

- We need to define the problem in terms of environment and agent

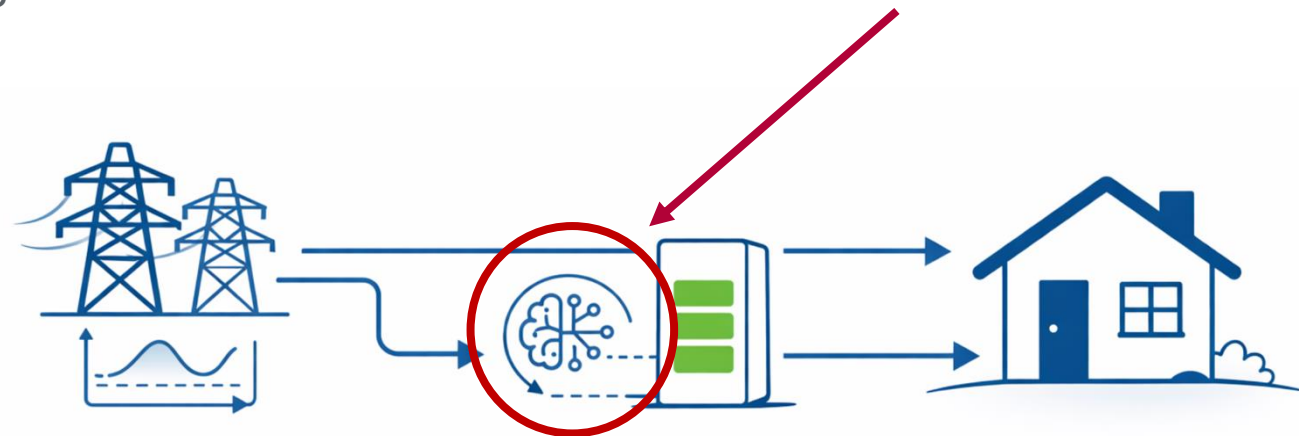


Environment

- Transition dynamics (discrete)
- Episode length
- Reward function
- Provides observation and reward to Agent

Agent

- Learns policy
- Chooses action based on observation

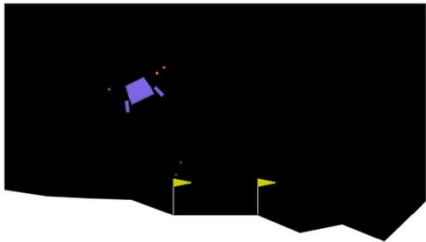


With funding from the:

Gymnasium



An API standard for reinforcement learning with a diverse collection of reference environments



- Gymnasium: standardized python API for RL environments
- Core functions:
 1. `reset()` – Resets the env to its initial state, returns the first observation
 2. `step()` – execute action, return observation, reward and info
- Optional methods: rendering, seeding, wrappers
- Can be used with simulators like Atari, MuJoCo, ...
- Alternatives and Related RL Environment Ecosystems:
 - PettingZoo (MARL)
 - Melting Pot (Benchmark for MARL experiments)

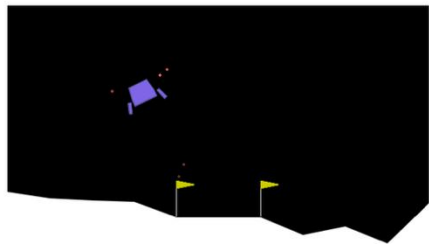
With funding from the:



Take a look at the Docs



An API standard for reinforcement learning with a diverse collection of reference environments



With funding from the:



Federal Ministry
of Research, Technology
and Space

gymnasium.farama.org

Gymnasium Documentation

Search

INTRODUCTION

Basic Usage

Training an Agent

Create a Custom Environment

Recording Agents

Speeding Up Training

Gym Migration Guide

API

Env

Make and register

Spaces

Wrappers

Vectorize

Utility functions

Functional Env

ENVIRONMENTS

Classic Control

Box2D

Env

class gymnasium.Env

[source]

The main Gymnasium class for implementing Reinforcement Learning Agents environments.

The class encapsulates an environment with arbitrary behind-the-scenes dynamics through the `step()` and `reset()` functions. An environment can be partially or fully observed by single agents. For multi-agent environments, see PettingZoo.

The main API methods that users of this class need to know are:

- `step()` - Updates an environment with actions returning the next agent observation, the reward for taking that actions, if the environment has terminated or truncated due to the latest action and information from the environment about the step, i.e. metrics, debug info.
- `reset()` - Resets the environment to an initial state, required before calling step. Returns the first agent observation for an episode and information, i.e. metrics, debug info.
- `render()` - Renders the environments to help visualise what the agent see, examples modes are "human", "rgb_array", "ansi" for text.
- `close()` - Closes the environment, important when external software is used, i.e. pygame for rendering, databases

Environments have additional attributes for users to understand the implementation

- `action_space` - The Space object corresponding to valid actions, all valid actions should be contained within the space.
- `observation_space` - The Space object corresponding to valid observations, all valid observations should be contained within the space.

ON THIS PAGE

Env

Methods

Env.step()

Env.reset()

Env.render()

Env.close()

Attributes

Env.action_space

Env.observation_space

Env.metadata

Env.render_mode

Env.spec

Env.unwrapped

Env.np_random

Env.np_random_seed

Implementing environments

Creating environments

Break 

With funding from the:



Now it's your turn

workshop-rl2-implementation

EXPLORER

WORKSHOP-RL2-IMPLEMENTATION

- > .claude
- > .github
- > .pytest_cache
- > .venv
- > 00_aisc
- > 01_workshop
 - > __pycache__
 - > envs
 - > __pycache__
 - __init__.py
 - battery_env.py M
- > notebooks
 - > figures
 - 00_explore_data.ipynb
 - 01_design_heuristi... M
 - 02_implement_env... M
 - 03_train_rl_withou... M
 - 04_train_rl_with_b... M
 - __init__.py
 - utils.py
 - > 02_solutions
 - > 03_data
 - > info-material

02_implement_environment.ipynb M

Generate + Code + Markdown | Run All ... workshop-rl2-implementation (Python 3.13.7)

Your Task

Open the file `01_workshop/envs/battery_env.py` and implement the 4 methods below the line:

```
# =====
# METHODS FOR PARTICIPANTS TO IMPLEMENT
# =====
```

Everything above that line is already implemented for you (data loading, action/observation spaces, forecasting, degradation model).

Implementation Order

#	Method	What it does
1	<code>_calculate_reward()</code>	determines the reward given to the agent
2	<code>_get_obs()</code>	Build the observation array
3	<code>reset()</code>	Initialize a new episode
4	<code>step()</code>	Process one action and update state

After implementing each method, come back to this notebook and run the corresponding test cell.

Let's implement the solution
together

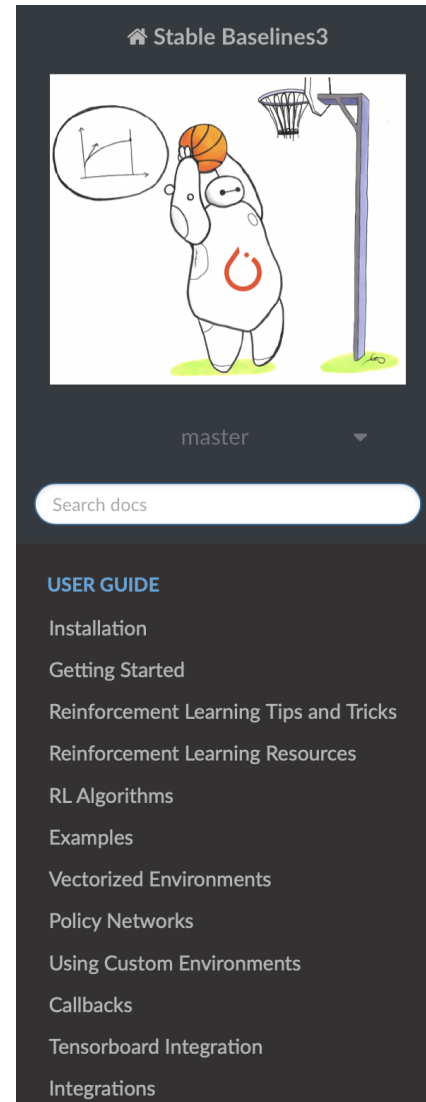
With funding from the:



Algorithms: Use Sb3

<https://stable-baselines3.readthedocs.io>

<https://github.com/DLR-RM/stable-baselines3>



🏠 / Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations [View page source](#)

Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations

Stable Baselines3 (SB3) is a set of reliable implementations of reinforcement learning algorithms in PyTorch. It is the next major version of **Stable Baselines**.

Github repository: <https://github.com/DLR-RM/stable-baselines3>

Paper: <https://jmlr.org/papers/volume22/20-1364/20-1364.pdf>

RL Baselines3 Zoo (training framework for SB3): <https://github.com/DLR-RM/rl-baselines3-zoo>

RL Baselines3 Zoo provides a collection of pre-trained agents, scripts for training, evaluating agents, tuning hyperparameters, plotting results and recording videos.

SB3 Contrib (experimental RL code, latest algorithms): <https://github.com/Stable-Baselines-Team/stable-baselines3-contrib>

SBX (SB3 + Jax): <https://github.com/araffin/sbx>

Main Features

- Unified structure for all algorithms
- PEP8 compliant (unified code style)
- Documented functions and classes
- Tests, high code coverage and type hints
- Clean code
- Tensorboard support

With funding from the:



Federal Ministry
of Research, Technology
and Space

Choice of algorithm

Environment, Data & Computing Ressources	Task Structure	Actions, Rewards & Policy
Dynamics known? ➤ Yes: model-based RL or planning ➤ No: model-free RL Training stability or sample efficiency? ➤ Sample efficiency: off-policy ➤ Training stability: on-policy Computing Ressources? ➤ Yes: Distributed training ➤ No: Not distributed	Complex subtasks? ➤ Yes: Hierarchical RL Expert demonstrations? ➤ Yes: Imitation learning Past information needed? ➤ Yes: Recurrent RL Long episodes? ➤ Yes: Value Estimates	Small discrete actions? ➤ Yes: Value methods Large, continuous actions? ➤ Yes: policy, actor-critic methods Sparse or deceptive rewards? ➤ Yes: Stochastic policy Risk-sensitive decisions? ➤ Yes: distributional RL Clear best action? ➤ Yes: deterministic policy possible

<https://rl-picker.github.io>

PPO and SAC

Learning Material

OpenAI – Spinning up - PPO
<https://spinningup.openai.com/en/latest/algorithms/ppo.html>

OpenAI – Spinning up – SAC
<https://spinningup.openai.com/en/latest/algorithms/sac.html>

Stanford RL 2025 – lecture 5
<https://www.youtube.com/watch?v=cRGKc-nAWho>

PPO	SAC
• Model-free • On-policy • Distributed training • Suited for long episodes, critic provides value estimates • Suited for continuous actions • Stochastic policy • Not risk-sensitive • High training stability • Lower sample efficiency ➤ Best when stability and simplicity matters, needs more data	• Model-free • Off-policy • Distributed training • Suited for long episodes, critic and replay • Suited for continuous actions • Stochastic policy • Not risk-sensitive • Stable training • High sample efficiency, reuses experience ➤ Best when data is expensive and exploration is important, needs more tuning

With funding from the:

Is this a RL problem? (Part II)

When RL could be useful:

- **Simulation / safe exploration is feasible**

RL is data hungry and learns by trial-and-error!

- **Sequential decision-making**

Decisions affect the environment, requiring long-term planning

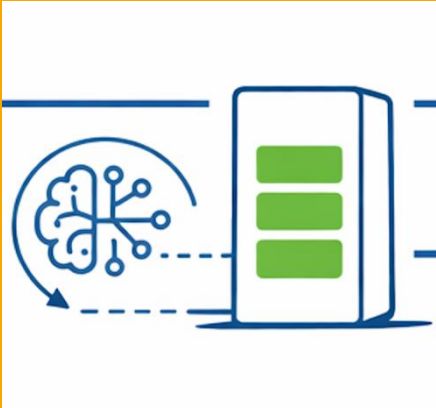
- **Complex or unknown dynamics**

System is too difficult to optimize analytically

→ Now we introduce a **battery degradation** mechanic into the environment!

With funding from the:

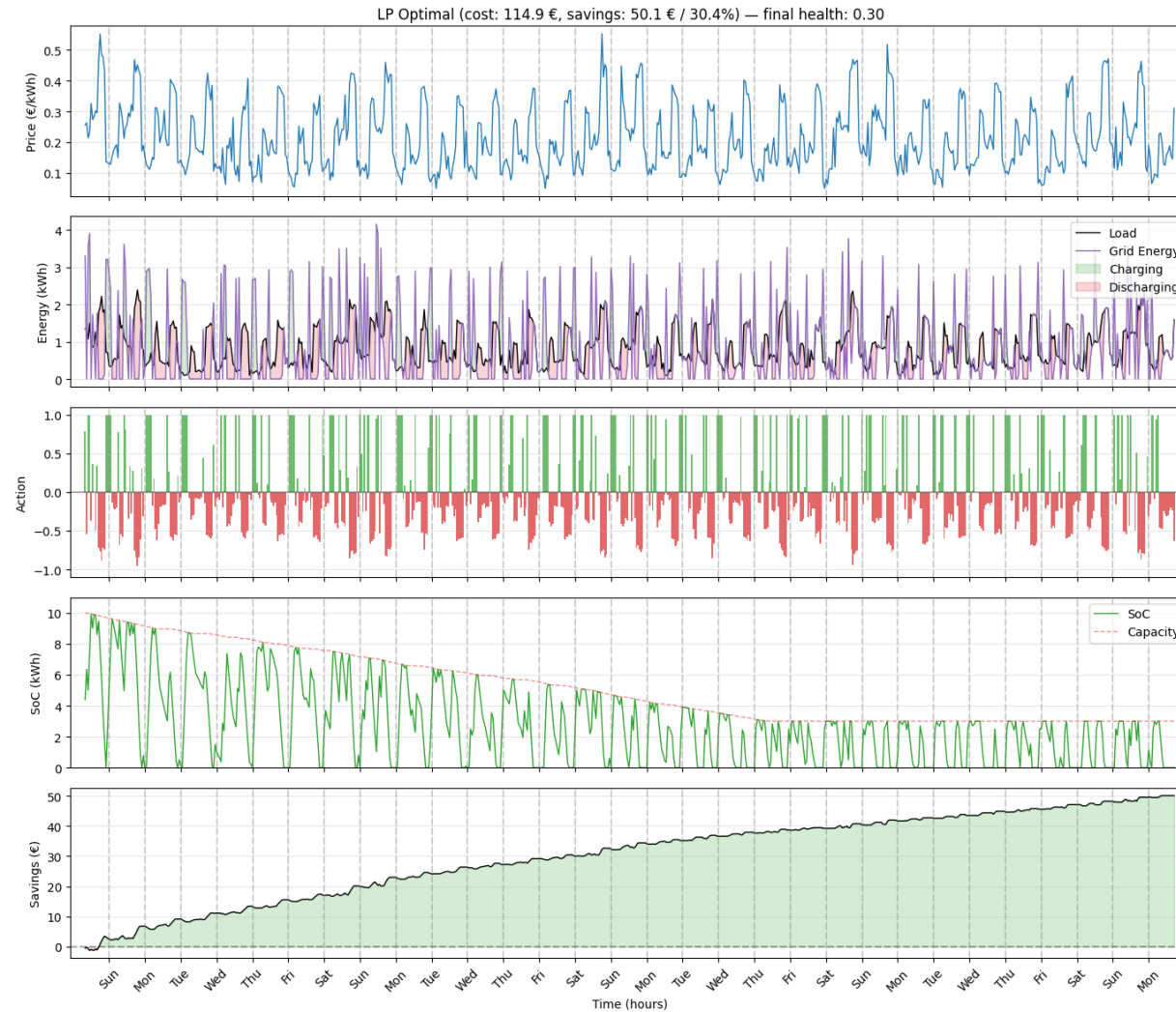
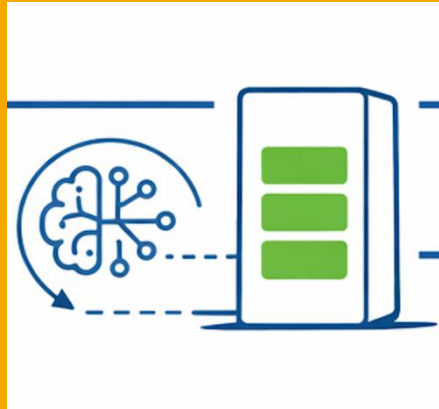
Battery Degradation



- Two sources of battery damage:
 - 1. Aggressive cycling:** Dis/charging above 70% of max power causes.
 - 2. Extreme SoC:** Keeping the battery below 20% or above 80% of capacity stresses the cells.
- Damage rate of 0.002
- Health starts at 1.0 and can't drop below 0.3.
- **Capacity** shrinks, making the battery less useful over time.
- Logic is implemented in ``_apply_degradation()``

With funding from the:

04_train_rl_with_battery_degradation.ipynb



With funding from the:

Discussion

- When to consider RL:
 - You only need to define *what* you want (reward), not *how* to get there
 - Simulation and reward shaping could be easier than constraint engineering
 - High-dimensional state/action spaces (Go)
- MPC and heuristic policies are still industry standard because they are interpretable and provide safety guarantees
- RL is deployed at scale primarily by tech giants (SIEMENS) with massive engineering resources and access to simulators or huge data in combination with other techniques.
- Nonetheless, RL provides a powerful framework.
There is a reason why DeepMind started with RL.

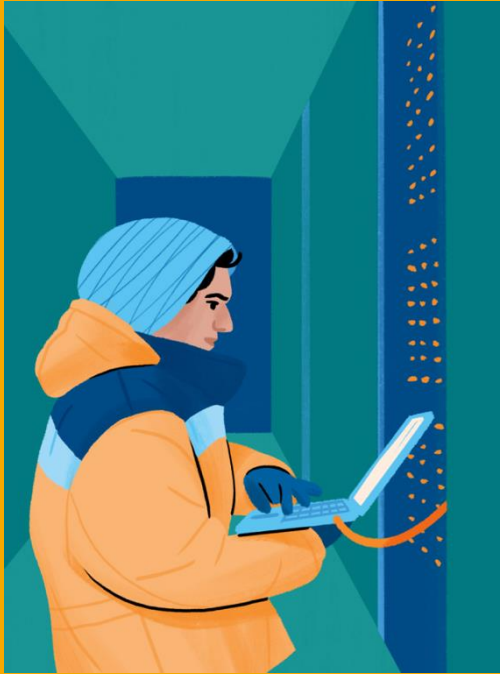
Real life example (1/2)

Meta Data Center Cooling

Application: Applying RL to **optimize airflow and cooling control**, aiming to reduce energy and water consumption

Why is it a RL Problem: decisions are sequential, uncertain, and affect long-term performance

System design in practice: RL trains offline in simulator proposes control actions, while physics-based models predict system response and safety rules enforce operational limits.



With funding from the:

Real life example (2/2)

Uber Marketplace Optimization

Application: RL improves rider-driver balance to reduce wait times and increase efficiency.

Why is it a RL Problem: Decisions affect future supply and demand, requiring long-term optimization under uncertainty.

System design in practice: Value-based RL suggests actions (e.g., incentives or repositioning), while the dispatch system enforces rules and performs final matching.



Figure 3: Trajectory of a driver through completing multiple trips.

With funding from the:

Conclusion

- Not every problem needs RL! Consider alternatives first.
- RL shines when: simulator available, complex dynamics, sequential decisions with long-term consequences
- Designing the environment is the real work, the algorithm is the easy part.
- Reward shaping lets you express complex objectives
- Most promising real-world approach: combine RL with other techniques

With funding from the:

Learning Materials

- Stanford CS224R
- OpenAI Spinning up
- Sb3 documentation
- Sutton and Barto book



Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 1: Class Intro
Stanford Online • 62K views • 2 months ago



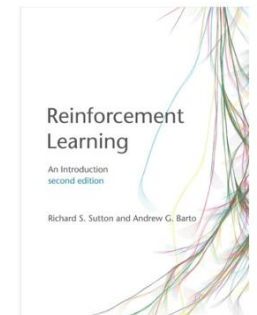
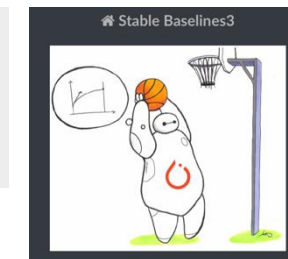
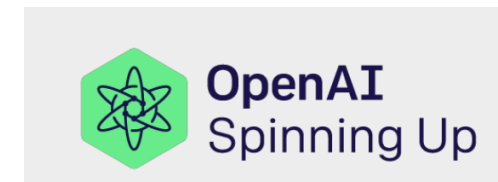
Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 2: Imitation Learning
Stanford Online • 13K views • 2 months ago



Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 3: Policy Gradients
Stanford Online • 8.4K views • 2 months ago



Stanford CS224R Deep Reinforcement Learning | Spring 2025 | Lecture 4: Actor-Critic Methods
Stanford Online • 6.8K views • 2 months ago



Links:

- <https://cs224r.stanford.edu>
- https://www.youtube.com/playlist?list=PLoROMvodv4rPwxEOONYRa_itZFdaKCylL
- <https://spinningup.openai.com/en/latest/index.html>
- <https://stablebaselines3.readthedocs.io/en/master/index.html>
- <http://incompleteideas.net/book/RLbook2020.pdf>

kisz@hpi.de

hpi.de/kisz

Your opinion is
important!



QR code for the
feedback form

Jill.barvencik@hpi.de

David.goll@hpi.de



With funding from the:

Learning Materials

Proximal Policy Optimization (PPO)

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ

2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data

3. Evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} w_n (\sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n}))$

4. Update policy with M gradient steps on surrogate objective:

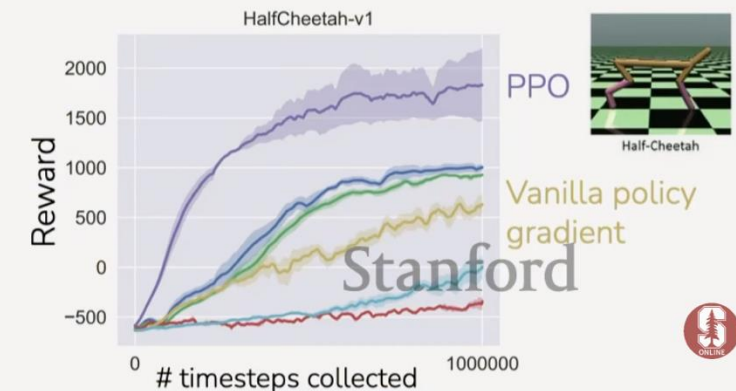
$$\tilde{J}(\theta') \approx \sum_{t,i} \min \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})} \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}), \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) \right)$$

Some example hyperparameters:

~2000 timesteps in batch of data

~10 epochs when updating policy clipping range $\epsilon=0.2$
($M \sim 300$ gradient steps with batch size 64)

~500 iterations $\rightarrow$ 1M total timesteps of experience



Choice of algorithm

Learning Material

OpenAI – Spinning up - PPO
<https://spinningup.openai.com/en/latest/algorithms/ppo.html>

OpenAI – Spinning up – SAC
<https://spinningup.openai.com/en/latest/algorithms/sac.html>

Stanford RL 2025 – lecture 5
<https://www.youtube.com/watch?v=cRGKc-nAWho>

PPO (Proximal Policy Optimization)

- Policy gradient method: directly learns a policy instead of value estimates
- Core idea: limits how much the policy can change each step
- Pro: „Works out of the box“, reliable baseline
- Con: On-policy, cannot reuse old data → sample-inefficient

SAC (Soft Actor Critic)

- Core idea: Maximize reward *and* entropy: explicitly incentivizes exploration
- Off-policy: Can reuse past experience from a replay buffer, making it much more sample-efficient than PPO

With funding from the: